東北大學 国家示范性软件学院
SOFTWARE COLLEGE OF NORTHEASTERN UNIVERSITY

Programming in Python

—— PART I Basics

1

Chapter 2: Variables and Data Types

Contents

Python Data Types

- **Python Simple Data Types** (简单数据类型)
  1. **Numbers** (数值类型)
  2. **Strings** (字符串)
  3. **Boolean** (布尔类型)
  4. **None** (空值)

- **Sequence Data Types** (序列数据类型)

  Python **built-in sequence types are**:

  - **lists**(列表), **tuples**(元组), **dictionaries**(字典), **and sets**(集合)

Python Data Types

1. Numbers

- There are three types of numeric literals:
  - integers
  - floating point numbers
  - imaginary numbers（虚数）
- There are no complex literals (complex numbers can be formed by adding a real number and an imaginary number)

Python Data Types

1. Numbers

- **Integers:** You can +, -, *, / integers in Python.

```
>>> 2 + 3
5
>>> 3 - 2
1
>>> 2 * 3
6
>>> 3 / 2
1.5
```

```
>>> 3 ** 2
9
>>> 3 ** 3
27
>>> 10 ** 6
1000000
```

```
>>> 2 + 3*4
14
>>> (2 + 3) * 4
20
```

- Python使用两个乘号表示乘方运算;
- Python还支持运算次序，因此你可在同一个表达式中使用多种运算

- There is **no limit** for the length of integer literals apart from what can be stored in available memory.

Python Data Types

1. Numbers

- **Floats:** Python calls any number with a decimal point a *float*.

```
>>> 0.1 + 0.1
0.2
>>> 0.2 + 0.2
0.4
>>> 2 * 0.1
0.2
>>> 2 * 0.2
0.4
```

```
>>> 0.2 + 0.1
0.30000000000000004
>>> 3 * 0.1
0.30000000000000004
```

- 从很大程度上说，使用浮点数时都无需考虑其行为。你只需输入要使用的数字，Python通常都会按你期望的方式处理它们；
- 但需要注意的是，结果包含的小数位数可能是不确定的

Python Data Types

1. Numbers

- Imaginary literals are described by the following lexical definitions词法定义:
  - imagnumber ::= (floatnumber | digitpart) ("j" | "J")
- Some examples of imaginary literals:
  - 3.14j 10.j 10j .001j 1e100j 3.14e-10j 3.14_15_93j
- Complex numbers are represented as a pair of floating point numbers and have the same restrictions on their range.
- To create a complex number with a nonzero real part, add a floating point number to it, e.g., (3.5 + 4.2j).

■ 一个虚数也可以看成实部为0.0的复数

Python Data Types

1. Numbers

- Python 中 int、float、bool、complex 都属于**不可变对象**（immutable）
- **不可变：**对象本身的内存里存储的值，不能原地修改。即，如果改变数值类型数据的值，将重新分配内存空间。

In [2]:

```
1 a=1
2 print(id(a))
3 a=2
4 print(id(a))
```

```
140734423050032
140734423050064
```

In [4]:

```
1 b=[1, 2, 4, 5]
2 print(id(b))
3 b[0]=0
4 print(b)
5 print(id(b))
```

```
2616202309440
[0, 2, 4, 5]
2616202309440
```

Python Data Types

2. Strings

- A *string* is a series of characters. Anything inside **quotes** 引号 is considered a string in Python.

■ Python不支持字符类型，单字符在Python中也是作为一个字符串使用。

- You can use single or double quotes around your strings like this:

```
'I told my friend, "Python is my favorite language!"'
"The language 'Python' is named after Monty Python, not the snake."
"One of Python's strengths is its diverse and supportive community."
```

■ 单引号和双引号的灵活使用，能够让你在字符串中包含引号和撇号

Strings

(1) Create and Access Strings

- Create a *String*: assign a value to a variable:

```
>>> var1 = 'Hello World!'
>>> var2 = "Python Programming "
```

- Access *SubStrings*: use square brackets to truncate(截取) a string:

```
>>>print("var1[0]:",var1[0])          # 取索引0的字符，注意索引从0开始
var1[0]: H
>>>print("var2[1:5]:", var2[1:5])     #切片（前闭后开）
var2[1:5]: ytho
```

Strings

(1) Create and Access Strings

- 字符串类型的数据也**属于不可变对象**（immutable）。即，如果改变字符串类型数据的值，将重新分配内存空间。

```
1 a='ABC'
2 a[0]='a'
```

```
TypeError                                 Traceback (most recent call
<ipython-input-10-a32382d8a459> in <module>
      1 a='ABC'
----> 2 a[0]='a'

TypeError: 'str' object does not support item assignment
```

```
1 a='ABC'
2 print(a)
3 print(a[0])
4 print(id(a))
5 a = 'aBC'
6 print(id(a))
```

```
ABC
A
2616135449584
2616203210416
```

Strings

(2) Python Escape Character (转义字符)

Escape Character	Description	Escape Character	Description
\ (在行尾时)	Backslash and newline ignored 续行符	\n	ASCII Linefeed (LF) 换行
\\	Backslash (\) 反斜杠符号	\v	ASCII Vertical Tab(VT) 纵向制表符
\'	Single quote (') 单引号	\t	ASCII Horizontal Tab(TAB)横向制表符
\"	Double quote (") 双引号	\r	ASCII Carriage Return (CR) 回车
\a	ASCII Bell (BEL) 响铃	\f	ASCII Formfeed (FF) 换页
\b	ASCII Backspace (BS) 退格	\000	Null 空
\oyy	Character with octal value yy 八进制数，yy代表的字符	\xyy	Character with hex value yy 十六进制数，yy代表的字符

Strings

(3) Python String Operators

suppose a = 'Hello' b = 'Python'

Operators	Example	Description	Operators	Example	Description

+	>>> a+b
	HelloPython
		字符串连接	join	>>> c = '+'
				>>> c.join(a)
				'h+e+l+l+o'
					使用一个分隔符(或字符串)连接字符串中每个字符

*	>>> a*2 HelloHello	重复输出字符串	in	>>> 'H' in a True	成员运算符，返回布尔类型
[ ]	>>> a[1] e	通过索引获取字符串中字符	not in	>>> 'M' not in a True	成员运算符，返回布尔类型
[:]	>>> a[1:4] ell	截取字符串中的一部分	r 或 R	>>> print(r'\n prints \n') \n prints \n	原始字符串

Strings

(4) String Formatted Output

- Python supports formatted string output. Although this can use very complex expressions, the most basic usage is to insert a value into a **template** with a string formatter.

```
>>>print ("My name is %s, and my age is %d. " % ('Jerry', 31))
```

My name is Jerry, and my age is 31

- Python用一个**元组**将多个值传递给格式化的模板，每个值对应一个字符串格式符。
- 上例将 'Jerry' 插入到 %s 处，31插入到 %d 处。

Strings

(4) String Formatted Output

Symbol	Description
%c	格式化字符
%s	格式化字符串
%d	格式化十进制整数
%u	格式化无符号整型
%o	格式化八进制数
%x	格式化十六进制数
%X	格式化十六进制数（大写）
%f	格式化浮点数字，可指定小数点后的精度
%e	用科学计数法格式化浮点数
%p	用十六进制数格式化变量的地址

Strings

(4) String Formatted Output

```
>>> Num1 = 0xFF
>>> Num2 = 0xAB03
>>> print('转换成十进制分别为：%d和%d' % (Num1, Num2))
转换成十进制分别为：255和43779
>>> Num3 = 1200000
>>> print('转换成科学计数法为：%e' % Num3)
转换成科学计数法为：1.200000e+06
```

Strings

(5) Using Variables in Strings

➢ **f-strings**

```
1  name = 'Ana'
2  food = 'pizza'
3  count = 4
4
5  phrase1 = f'{name} eats only {food} on weekends'
6  print(phrase1)
7
8  phrase2 = f'{name} eats {count} {food}s each week'
9  print(phrase2)
```

■ fstring 是在字符串前面加上字母“f”或“F”，然后变量可以直接输入到字符串中的括号中进行替换。

```
Ana eats only pizza on weekends
Ana eats 4 pizzas each week
```

Strings

(5) Using Variables in Strings

- f-strings

■ 甚至可以在 { } 中放入 input 函数，让用户输入

```
>>>print(f"name: {input('Input name: ')} age: {input('Input age: ')} sex: {input('Input sex: ')}")
Input name: Jerry
Input age: 31
Input sex: female
name: Jerry age: 31 sex: female
```

Strings

(6) Changing Case in a String with Methods

```
name = "ada lovelace"
print(name.title())
```

```
Ada Lovelace
```

- title() 方法以首字母大写的方式显示每个单词

```
name = "Ada Lovelace"
print(name.upper())
print(name.lower())
```

```
ADA LOVELACE
ada lovelace
```

- 存储数据时，方法lower() 很有用。很多时候，你无法依靠用户来提供正确的大小写，因此需要将字符串先转换为小写，再存储它们。
- 以后需要显示这些信息时，再将其转换为最合适的大小写方式。

Strings

(7) Stripping Whitespace

- rstrip() method

```
>>> favorite_language = 'python '
>>> favorite_language
'python '
>>> favorite_language.rstrip()
'python'
>>> favorite_language
'python '
```

■ 该方法不改变原变量值，如果要永久删除空白，需要为变量重新赋值

```
>>> favorite_language = 'python '
>>> favorite_language = favorite_language.rstrip()
>>> favorite_language
'python'
```

Strings

(7) Stripping Whitespace

- rstrip(), lstrip(), strip()

```
>>> favorite_language = ' python '
>>> favorite_language.rstrip()
' python'
>>> favorite_language.lstrip()
'python '
>>> favorite_language.strip()
'python'
```

Boolean type

- A Boolean value is either True or False

- The Boolean type has the following operations:
  - and: True **and** True -->True
  - or: False **or** True --> True
  - not: **not** False -->True, **not** True --> False

Boolean type

- Boolean data can also perform *and, or, not* operations with other data types;

- Example:

```
>>> a = 'python'                # a为非空字符串，为True
>>> print (a and True)          # 结果是 True
True
>>> b = ''                      # b为空字符串，为False
>>> print (b or False)          # 结果是 False
False
```

Boolean type

- Other types of data are considered FALSE in the following situations:

```
>>> bool(0)
False
>>> bool(0.0)
False
>>> bool('')
False
>>> bool("")
False
>>> bool(None)
False
```

```
>>> bool(())
False
>>> bool([])
False
>>> bool({})
False
>>>
```

- 数值型的0，包括整形0和浮点型0.0
- 字符串型的空串，如 ‘’ 或 “”
- 序列类型的空元组()，空列表[]，空字典{}
- 表示空值的None

- All other values are TRUE

Boolean type

- Boolean values can be treated as numbers, and calculated with other numeric data.
  - True --> 1 False --> 0
- Example:

```
>>> x = False
>>> a = x+(5>4)        # 结果a是1
>>> a
1
>>> b = x+5            # 结果b是5
>>> b
5
```

None（空值）

- A *None* value is a special value in Python.
- It does not support any operations nor any built-in function methods.
- Comparing *None* with any other data type always returns *False*.
- Functions that do not specify a return value in Python automatically return *None*.

■ None 是 Python 的特殊常量，唯一取值就是 None，类型是 NoneType。它不是 0、不是 False、不是空字符串""。判断是否为 None，强烈建议用 is None / is not None，不要用 ==。

```
>>> print(type(None))      # <class 'NoneType'>
>>> print(None == 0)       # False
>>> print(None == "")      # False
>>> print(None == False)   # False
```

None（空值）

- ❌ 错误：不要用可变对象当默认参数，比如 def func(data=[])
- ✅ 经典用法：用None充当占位默认值，函数内部再创建可变对象

```
1  def add_item(item, data=[]):
2      data.append(item)
3      return data
4
5  print(add_item(1))
6  print(add_item(2))
7  my_list = [10,20]
8  print(add_item(99, my_list))
9
```

✓ [26] 29ms

```
[1]
[1, 2]
[10, 20, 99]
```

```
1  def add_item(item, data=None):
2      if data is None:
3          data = []
4      data.append(item)
5      return data
6
7  print(add_item(1))
8  print(add_item(2))
9  my_list = [10,20]
10 print(add_item(99, my_list))
11
```

✓ [27] 32ms

```
[1]
[2]
[10, 20, 99]
```

None（空值）

- 作为 “空 / 未赋值” 标记，区分 0、False、空
- 很多场景：0、""、False 是合法业务值，不能直接用if not x判断。

```python
def calc_offset(offset=None):
    if offset is None:
        offset = 20  # 用户没传参数，使用默认20
    # 如果 offset=0，是用户明确传0，就使用0，不会被当成空
    return offset

print(calc_offset())    #20
print(calc_offset(0))   #0
```

控制台

```
20
0
```

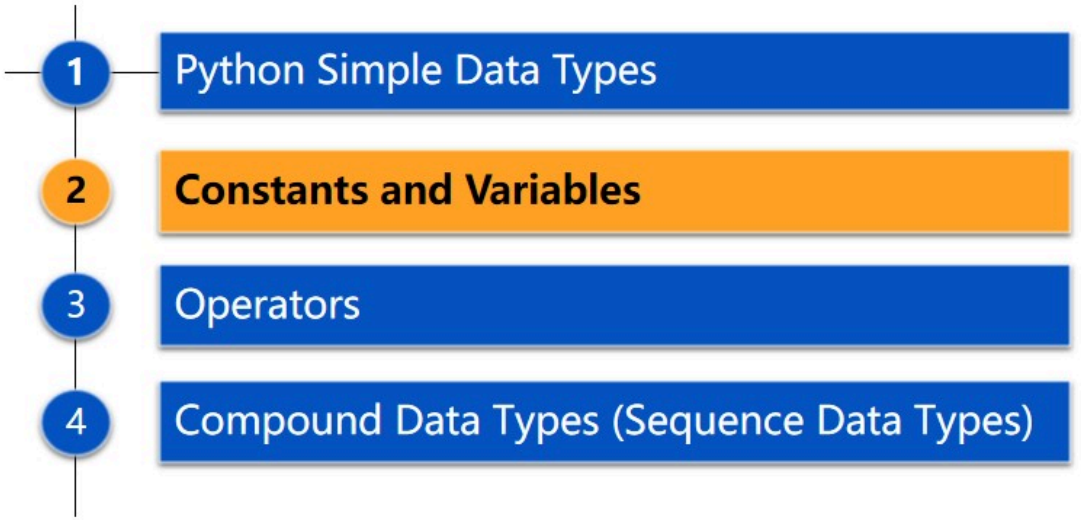
- 这里不能写 if not offset:，否则传入 0 也会走默认逻辑，业务出错。
- 妙用：None专门表示「参数未提供」，和合法的 0、False 做隔离。

Data Type Conversion

```
>>> x=20                    # 八进制为24
>>> y=345.6
>>> print(oct(x))           #将整数转换为八进制字符串
0o24
>>> print(int(y))           #转换为一个整数
345
>>> print(float(x))         #转换为浮点数
20.0
>>> print(chr(65))          #将一个整数ASCII转换为字符
A
>>> print(ord('B'))         # 将一个字符转换为它的ASCII码值
66
```

Chapter 2: Variables and Data Types

Contents



Constants and Variables

Variables

- In computer programs, variables can be not only numbers, but also any data type.
- Using variables in Python, you need to adhere to a few rules and guidelines.

```
>>> stu01 = 123          # 变量名只能包含字母、数字和下划线
>>> _t007 = 'T007'       # 变量名可以字母或下划线打头，但不能以数字打头
>>> Answer_T = True      # 变量名不能包含空格，但可使用下划线来分隔其中的单词
>>> listO = [1,2]        # 慎用小写字母l和大写字母O，因为它们可能被人错看成数字1和0
```

Constants and Variables

Variable definitions

```
>>> a = 123      # Python中用赋值语句来定义变量，如定义变量a是整数
>>> a = 'ABC'    # 同一个变量可以反复赋值，甚至可以是不同类型的变量，如a变为字符串
```

- A language in which the type of the variable itself **is not fixed** is called **dynamic language** 动态语言, and the corresponding language is **static language**.

  - In C language (static language)

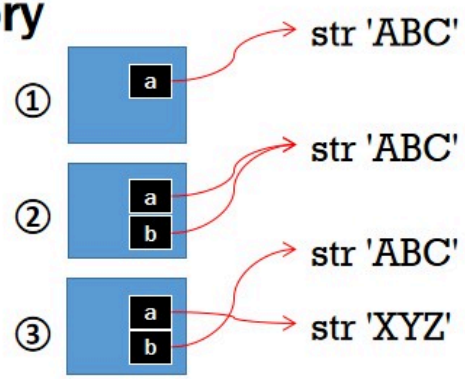
```
int a = 123;
a = 'ABC';
```

- **Dynamic languages are more flexible**(灵活) **than static languages**

Constants and Variables

Representation of Variables in computer memory

```
>>> a = 'ABC'      # ①
>>> b = a          # ② 可以把变量a赋值给另一个变量b
>>> a = 'XYZ'      # ③
>>> print(b)
ABC
```



■ 当定义变量a时，Python解释器做了两件事情：

（1）在内存中创建了一个'ABC'的字符串；

（2）在内存中创建了一个名为a的变量，并把它指向'ABC'

Constants and Variables

Constants

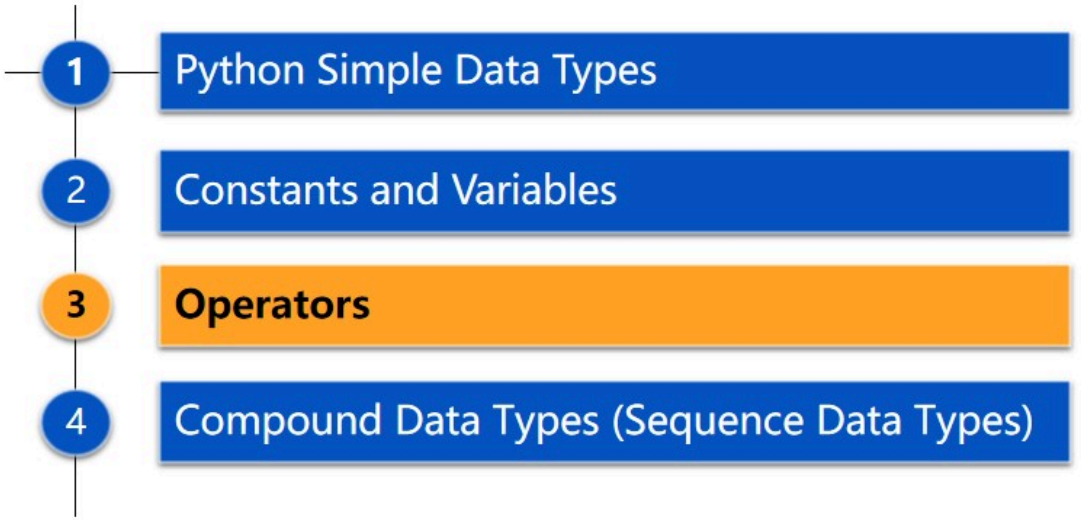
- Python doesn't have built-in constant types, but Python programmers use all capital letters to indicate a variable should be treated as a constant and never be changed. Eg:

  >>> PI = 3.14159265359

- But in fact PI is still a variable, and Python **has no mechanism** at all to guarantee(保证) that PI will not be changed.
- Therefore, using all uppercase variable names to represent constants is just a **habitual usage**, which can actually change the value of the variable.

Chapter 2: Variables and Data Types

Contents



Operators

- In a program, expressions are used to evaluate(计算求值), and expressions consist of operators and operands.
  - An operator (运算符) is a **symbol** that expresses a certain operation
  - Operands (操作数) include **constants**, **variables**, and **functions**

Operators

- The Python language supports the following types of operators:
  - Arithmetic operators (算术运算符)
  - Comparison operators (比较，即关系运算符)
  - Assignment operator (赋值运算符)
  - Logical Operators (逻辑运算符)
  - Bitwise operators (位运算符)
  - Member operator (成员操作符)
  - Identity operator (标识操作符)

} Python独有

Operators

1. Arithmetic Operators (算术运算符)

- Suppose the variables: **a=2, b=7**

Operators	Example	Description
+	a + b = 9	addition
-	a - b = -5	subtraction
*	a * b = 14	multiplication
/	b / a = 3.5	division，永远返回浮点数；C / Java：b/a = 3，截断取整
%	b%a = 1; -7%3=2	模/求余运算，Python 取模结果符号**和除数一致**；C 语言取模符号**和被除数一致**，行为不一样，-7%3=-1
**	a ** b =2^7	Exponentiation，幂运算；C / Java：没有幂运算符，需要调用库函数 pow()
//	9//2 = 4; 9.0//2.0 = 4.0	divide exactly，整除；**Python独有**

Operators

2. Relational Operator (关系运算符)

- Suppose the variables: a=10, b=20

Operators	Example	Description
==	(a==b) is False	Checks if the values of the two operands are equal, if so the result is True
!=	(a!=b) is True	if the values are not equal the result is True
<>	(a<>b) is True	if the values are not equal the result is True
>	(a>b) is False	Checks if the value of the left operand is greater than the right
<	(a<b) is True	Checks if the value of the left operand is less than the right
>=	(a>=b) is False	Checks if the value of the left operand is greater than or equal to the right
<=	(a<=b) is True	Checks if the value of the left operand is less than or equal to the value of the right, if so the result is True

- Python 支持**连续链式比较**，**C、Java 不支持**。

```
>>> 10 < x < 20        # Python 合法，等价于 10 < x and x < 20
```

- C、Java 不能写 10 < x <20；会被解析为 (10<x) <20，逻辑错误，必须写成 10<x && x<20

Operators

3. Logical Operators (逻辑运算符)

Operators	Example	Description
and	(True and True) is True	The result is True if both operands are true (non-zero)
or	(True or False) is True	The result is True if at least one of the two operands is true
not	not (True and True) is False	The result returns False if the operand is true, otherwise returns True

语言	逻辑与	逻辑或	逻辑非
Python	and	or	not
C/Java	&&	||	!

- Python 使用英文单词，C/Java 使用符号。
- 短路特性三者都一样：and 左边假直接短路；or 左边真直接短路。

- Python `and/or` 会返回实际运算对象，不一定返回布尔值；C/Java 的 `&& ||` 运算结果永远是布尔值 true/false。

```
>>> 3 and 5      # 返回5
>>> 0 or 10      # 返回10
```

Operators

4. Assignment operator (赋值运算符)

Operators	Example	Description
=	c = a a,b = 10,20 # 解包，交换变量 a,b = b,a 非常方便	direct assignment Python 支持链式赋值、多元解包赋值，C 支持链式赋值，但没有解包
+=	c += a is equivalent to c = c + a	Add first, then assign Python 没有 i++、i-- 自增自减运算符，只能 i+=1
-=	c -= a is equivalent to c = c - a	Subtract first, then assign
*=	c *= a is equivalent to c = c * a	Multiply first, then assign
/=	c /= a is equivalent to c = c / a	First do the division, then assign
%=	c %= a is equivalent to c = c % a	First do the modulo operation(求余) , then assign
**=	c **= a is equivalent to c = c ** a	Do exponentiation(幂运算) first, then assign
//=	c //= a is equivalent to c = c // a	First do the division exactly(整除), and then assign

Operators

4. Assignment operator（赋值运算符）

- **解包(unpacking)**：把一个**容器**里面的多个元素，一次性拆开，分别赋值给多个变量。容器可以是元组、列表、字符串等可迭代对象。

```
# 右边是元组 (10,20)，自动解包，分别给到a、b
a, b = 10, 20
print(a) #10
print(b) #20
```

等价于

```
t = (10, 20)
a, b = t    # 将元组t解包，拆分出两个值赋值给a,b
```

- **经典用法**：交换两个变量，不需要临时变量

```
a = 1
b = 2
a, b = b, a    # 解包交换，a=2,b=1
```

扩展解包：

*rest收集剩下全部元素。

```
nums = [1,2,3,4]
first, *rest = nums
print(first) #1
print(rest)  #[2,3,4]
```

Operators

4. Assignment operator（赋值运算符）

- 三元条件运算符语法不一样

python

```python
# Python:  真值 if 条件 else 假值
max_val = a if a>b else b
```

java

```java
// Java / C
max_val = a>b ? a : b;
```

Operators

5. Bitwise operators (位运算符)

- Suppose the variables: a = 0b00111100, b = 0b00001101

Operators	Example	Description
&	a & b is 0b00001100	Bitwise AND
|	a | b is 0b00111101	Bitwise OR
^	a ^ b is 0b00110001	Bitwise XOR
~	~ a is 0b1100 0011	Bitwise negation (取反)
<<	a << 2 is 0b11110000	Bitwise left shift operation
>>	a >> 2 is 0b00001111	Bitwise right shift operation

Operators

6. Member operator（成员运算符），Python独有

- Member operator determines whether there is a member in the sequence.

Operators	Example	Description
in	>>> 3 in [1,2,3,4] True	x in y, the result of the operation is True if x is a member of the sequence y, False otherwise
	>>> 5 in [1,2,3,4] False	(如果x是序列y的成员，则运算结果为True，否则为False)
not in	>>> 3 not in [1,2,3,4] False	x not in y, the result of the operation is True if x is not a member of the sequence y, otherwise False
	>>> 5 not in [1,2,3,4] True	(如果x不是序列y的成员，则运算结果为True，否则为False)

- C / Java：没有该运算符，需要手写循环 / 集合方法判断包含关系。

Operators

7. Identity operator（标识运算符），Python独有

- 判断两个对象是不是**同一个对象**（内存地址是否相同），不是判断值相等。

Operators	Example	Description (Suppose a=123, b='test')

is	>>> a is b
	False
	>>> b is 'test'
	True
		如果操作符两侧的变量指向相同的对象，则运算结果为True，否则为False

is not	>>> a is not b
	True
	>>> a is not 123
	False
		如果操作符两侧的变量指向相同的对象，则运算结果为False，否则为True

```
>>> a = 123
>>> b = 'test'
>>> id(a)
140349906483376
>>> id(123)
140349906483376
>>> id(b)
140349903986928
>>> id('test')
140349903986928
>>>
```

```
a = [1,2]
b = [1,2]
a == b    # True，值相等
a is b    # False，不是同一个对象
```

如何判断两个对象是相同的?

- Python使用内置函数 id(x)，查看x所在的内存地址
- C：用指针比较；Java：== 既做数值比较又做引用比较，容易混淆。没有专门的 is 关键字。

Operators

8. Operator Priority (运算符优先级)

Priority	Operator	Description
1	**	exponentiation (幂)
2	~, +, -	Negate, unary Positive and Negative
3	*, /, %, //	Multiply, Divide, Modulo, and Divide exactly
4	+, -	Addition, Subtraction
5	>>, <<	shift by bit
6	&	bitwise AND
7	^, |	Bitwise XOR, Bitwise OR
8	<=, <, >, >=	Comparison operators
9	<>, ==, !=	Comparison operators
10	=, %=, /=, //=, -=, +=, *=, **=	Assignment operators
11	is, is not	Identity operators
12	in, not in	Member operators
13	not, or, and	Logical operators

单选题 1分

在Python中，如何表示乘方运算?

A 使用一个乘号

B 使用两个乘号

C 使用三个乘号

D 使用一个加号

单选题 1分

如何创建一个字符串变量?

A 使用双引号或单引号赋值给变量

B 使用大括号赋值给变量

C 使用方括号赋值给变量

D 使用圆括号赋值给变量

单选题 1分

在Python中，如何访问字符串的第一个字符?

A 使用索引 `[-1]`

B 使用索引 `[0]`

C 使用索引 `[1]`

D 使用方法 `.first()`

主观题 2分

请解释使用title()和lower()方法来处理用户输入的字符串是什么效果，并说明其应用场景。

多选题 2分

在Python中，布尔值True和False可以进行哪些操作?

A 加法和减法

B 逻辑运算：and, or, not

C 字符串拼接

D 位运算

单选题 1分

在Python中，如何将整数20转换为八进制字符串？

A 使用oct()函数

B 使用int()函数

C 使用float()函数

D 使用chr()函数

单选题 1分

在Python中，如何表示一个变量应该被视为常量?

A 使用小写字母

B 使用下划线

C 使用全大写字母

D 使用数字

多选题 1分

Python支持以下哪些类型的运算符?

A 算术运算符

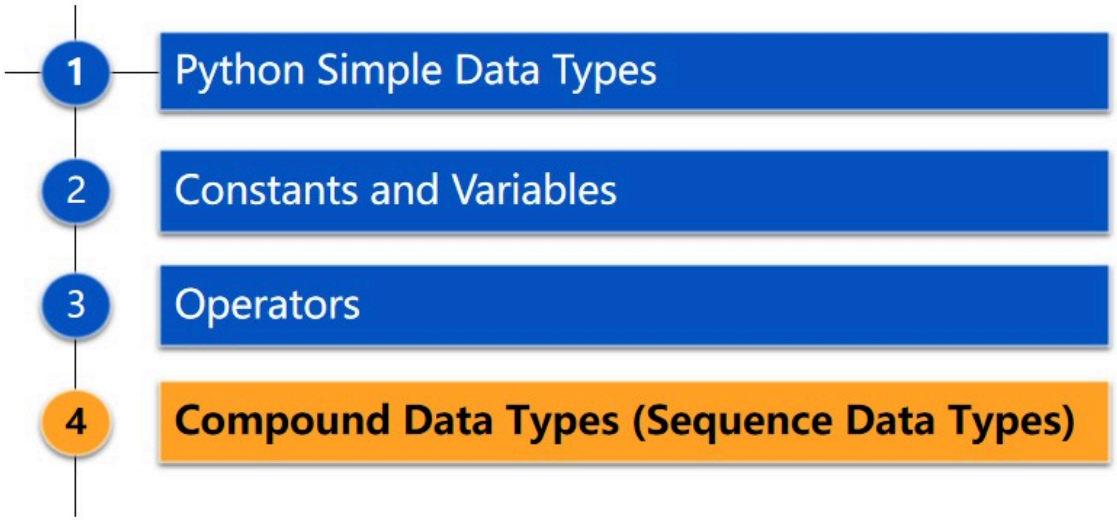
B 关系运算符

C 逻辑运算符

D 成员运算符

Chapter 2: Variables and Data Types

Contents



Sequence Data Types

- The most common built-in sequence types in Python are lists, tuples, dictionaries, and sets.
  - The data elements of a *list*, a *tuple* and a *string* are **ordered**;
    - Each element is assigned a number as its position or index, the first index is 0, the second index is 1, and so on...
    - Can perform **indexing**, **truncation** (slicing切片), **addition**, **multiplication**, **member checking**
  - Data elements in a *dictionary* and a *set* are **unordered**, and data elements cannot be indexed by position.
  - In addition, Python has built-in methods to get the length, maximum and minimum elements of sequence type data.

补充内容

- 严格来说：list列表、tuple元组是**序列**；dict字典是**映射**；set集合是**无序集合**，很多教材会把这四个放在一起讲解（我们也放在一起讲解） 
- **Python**：list、tuple、dict、set 全部语言内置(build-in)，**无需 import**
- **C语言**：**没有内置容器**，全部靠数组、结构体、指针手动实现；标准库<stdlib.h>提供动态数组、链表**需要自己封装**
- **Java**：需要导入集合框架java.util：ArrayList、LinkedList、HashMap、HashSet、ArrayDeque等，**不是语言原生关键字，是类**

补充内容

1. list 列表（**可变mutable**序列，对应不可变对象immutable）

```python
lst = [1,2,"hello",True] # 可以存放不同类型元素，动态扩容
lst.append(3)
lst[0] = 100
```

- **Python底层**：动态数组，自动扩容；元素类型可以混合；支持切片lst[1:3]；负下标lst[-1]取末尾。
- **Java 对应**：ArrayList
  - 只能存对象，不能直接存基础类型 int（要用 Integer 包装类）；泛型约束类型，**默认不允许随便混存不同类型。**
- **C语言**：没有直接等价，手动写动态数组，自己管理 realloc 扩容，手动管理内存。
-
- > 核心差异：

补充内容

1. list 列表（**可变mutable**序列，对应不可变对象immutable）

```python
lst = [1,2,"hello",True] # 可以存放不同类型元素，动态扩容
lst.append(3)
lst[0] = 100
```

核心差异：

1. Python list 原生支持异构元素；Java ArrayList 一般用泛型限定同类型；C 数组全部同类型。
2. Python 切片语法**a[start:end:step]**是语言层面特性，Java/C 没有，需要手写循环。
3. Python 直接lst[-1]访问尾部，Java/C 必须计算下标。

Lists

What Is a List?

- A *list* is a collection of items in a particular order（特定顺序）.
- Lists can be indexed, sliced and manipulated with other built-in functions.
- You can put anything you want into a list.

- Here's a simple example of a list:

```
>>> list1 = ['flower', 'food', 1997, 15.8];
>>> list2 = [1, 2, 3, 4, 5 ];
```

Lists are similar to arrays in other languages, but much more powerful than arrays.

Lists

1. Accessing Elements in a List

- By the position, or *index* of the item desired:

```
>>> list1 = ['flower', 'food', 1997, 15.8];
>>> list2 = [1, 2, 3, 4, 5, 6, 7 ];

>>> print (list1[0] )          # Access only one element
flower
>>> print (list2[1:5] )# Access multiple elements
[2, 3, 4, 5]                   # slicing returns a new list
>>> print (list1[-1] )         # Accessing the last element in a list
15.8
```

```
>>> list1 = [0,1,2,3,4,5]
>>> print(list1[3:5])
[3, 4]
>>> print(list1[3:6])
[3, 4, 5]
>>> print(list1[3:7])
[3, 4, 5]
>>> print(list1[-1])
5
>>> print(list1[-1:-3])
[]
>>> print(list1[-3:-1])
[3, 4]
>>> print(list1[0:6:2])
[0, 2, 4]
>>> print(list1[-1:-6:-1])
[5, 4, 3, 2, 1]
>>> print(list1[-1:-7:-1])
[5, 4, 3, 2, 1, 0]
```

Lists

2. Modifying Elements in a List

```
>>> motorcycles = ['honda', 'yamaha', 'suzuki']
>>> print(motorcycles)
['honda', 'yamaha', 'suzuki']

>>> motorcycles[0] = 'ducati'          # 意大利摩托车生产商杜卡迪
>>> print(motorcycles)
['ducati', 'yamaha', 'suzuki']
```

Lists

3. Adding Elements to a List

- Appending Elements to the End of a List:

```
>>> motorcycles = ['honda', 'yamaha', 'suzuki']
>>> print(motorcycles)
['honda', 'yamaha', 'suzuki']

>>> motorcycles.append('ducati')          # 使用列表的append()方法，增加元素至列表尾部
>>> print(motorcycles)
['honda', 'yamaha', 'suzuki', 'ducati']
```

Lists

3. Adding Elements to a List

- Inserting Elements into a List:

```
>>> motorcycles = ['honda', 'yamaha', 'suzuki']


>>> motorcycles.insert(0, 'ducati')    # insert()方法，增加元素至列表指定位置
>>> print(motorcycles)
['ducati', 'honda', 'yamaha', 'suzuki']
>>> motorcycles.insert(3, 'test')                # insert()方法，增加元素至列表指定位置
>>> print(motorcycles)
['ducati', 'honda', 'yamaha', 'test', 'suzuki']
```

Lists

4. Removing Elements from a List

- Removing an Item Using the del Statement

```
>>> motorcycles = ['honda', 'yamaha', 'suzuki']
>>> del motorcycles[0]          # 使用Python内置函数del，删除列表指定元素
>>> print(motorcycles)
['yamaha', 'suzuki']

>>> del(motorcycles[1])         # 使用Python内置函数del，删除列表指定元素
>>> print(motorcycles)
['yamaha']
```

Lists

4. Removing Elements from a List

- Removing an Item Using the pop() Method

```
>>> motorcycles = ['honda', 'yamaha', 'suzuki']


>>> popped_motorcycle = motorcycles.pop()      # 使用列表的pop()方法，删除列表末尾元素
>>> print(motorcycles)
['honda', 'yamaha']
>>> print(popped_motorcycle)
['suzuki']
```

Lists

4. Removing Elements from a List

- Popping Items from any Position in a List

```
>>> motorcycles = ['honda', 'yamaha', 'suzuki']


>>> first_owned = motorcycles.pop(0)        # pop()方法可删除列表中任何位置的元素
>>> print(first_owned)
honda
>>> print(motorcycles)          # 每当使用pop()时，被弹出的元素就不再在列表中了
['yamaha', 'suzuki']
```

Lists

4. Removing Elements from a List

- Removing an Item by Value

```
>>> motorcycles = ['honda', 'yamaha', 'suzuki', 'ducati']


>>> motorcycles.remove('ducati')   # 使用remove()方法来删除指定值的元素
>>> print(first_owned)
honda
>>> print(motorcycles)
['honda', 'yamaha', 'suzuki']
```

- 方法remove() 只删除第一个指定的值。
- 如果要删除的值可能在列表中出现多次，就需要使用循环来判断是否删除了所有这样的值。

Lists

5. Making Numerical Lists

- Many reasons exist to store a set of numbers.
- Lists are ideal for storing sets of numbers
- Python provides a variety of tools to help you work efficiently with lists of numbers.
- Example:

class range(*stop*)

class range(*start*, *stop*[, *step*])

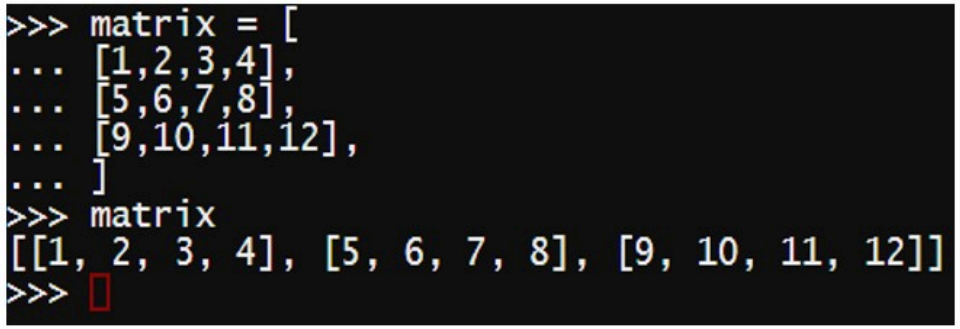
```
>>> list(range(10))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> list(range(1, 11))
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> list(range(0, 30, 5))
[0, 5, 10, 15, 20, 25]
>>> list(range(0, 10, 3))
[0, 3, 6, 9]
>>> list(range(0, -10, -1))
[0, -1, -2, -3, -4, -5, -6, -7, -8, -9]
>>> list(range(0))
[]
>>> list(range(1, 0))
[]
```

Lists

5. Create a Multidimensional list/Nested List (多维列表/嵌套列表)

- A 3×4 matrix implemented as a list of 3 lists of length 4:

```
>>> matrix = [
...     [1, 2, 3, 4],
...     [5, 6, 7, 8],
...     [9, 10, 11, 12],
... ]
>>> matrix
[[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]
```



- A 2-D list has one more index than a 1-D list, and the elements can be obtained as follows:

```
>>> matrix[1][3]    # 多一个索引来访问某个元素
8
```

Lists

Example: Define a list with 3 rows and 6 columns, and print out the element values

```
rows=3
cols=6
matrix = [ [0 for col in range(cols)] for row in range(rows)]# 列表推导式
for i in range(rows):
        for j in range(cols):
            matrix[i][j]=i*3+j
            print (matrix[i][j],end=",")
        print ('\n')
```

```
0,1,2,3,4,5,
3,4,5,6,7,8,
6,7,8,9,10,11,
```

补充：List/set/dict comprehensions

- Comprehensions（推导式/解析式） are a unique feature of Python.
- A comprehension is a structure that can construct a new sequence data from a sequence data.
- There are three types of Comprehensions：
  - List Comprehensions（列表推导式）
  - dict Comprehensions（字典推导式）
  - set Comprehensions（集合推导式）

补充：List comprehensions

- List comprehensions provide a concise(简洁的) way to create lists.

```
variable = [out_exp_res for out_exp in input_list if out_exp == 2]
```

- out_exp_res: 列表生成元素表达式，可以是有返回值的函数
- for out_exp in input_list: 迭代input_list，将out_exp传入out_exp_res表达式中
- if out_exp == 2: 根据条件过滤哪些值被迭代

- Example 1:

```
>>> squares = [x**2 for x in range(10)]
>>> print(squares)
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

补充：List comprehensions

- List comprehensions provide a concise(简洁的) way to create lists.

```
variable = [out_exp_res for out_exp in input_list if out_exp == 2]
```

- Example 2:

```
>>> def squared(x):
...     return x*x
...
>>> multiples = [squared(i) for i in range(30) if i % 3 == 0]
>>> multiples
[0, 9, 36, 81, 144, 225, 324, 441, 576, 729]
```

Working with Lists

List Operator

Operators	Example	Description
+	>>> [1, 2, 3] + [4, 5, 6] [1, 2, 3, 4, 5, 6]	list combination (列表组合)
*	>>> ['Hi!'] * 4 ['Hi!', 'Hi!', 'Hi!', 'Hi!']	list repeats (列表重复)
in	>>> 3 in [1, 2, 3] True	List member operations (元素是否存在于列表中)
in	>>> for x in [1, 2, 3]: ... print(x, end=" ") ... 1 2 3	iterative operation (迭代操作)

Working with Lists

Built-in functions for list

Functions	Example	Description
len(list)	>>> len([1, 2, 3, 4, 5]) 5	Return the length (the number of items) of a list
max(list)	>>> max([1, 2, 3, 4, 5]) 5	Return the largest item in an iterable (list)
min(list)	>>> min([1, 2, 3, 4, 5]) 1	Return the smallest item in an iterable (list)

list(seq)	>>> list((1, 2, 3, 4, 5))
	[1, 2, 3, 4, 5]
	>>> list({1, 2, 3, 4, 5})
	[1, 2, 3, 4, 5]
	>>> list({'a':1, 'b':2, 'c':3})
	['a', 'b', 'c']
		Convert other sequence type data to list

Working with Lists

Methods of list

Methods	Description
list.append(obj)	在列表末尾添加新的对象
list.insert(index, obj)	将对象插入列表指定位置
list.extend(seq)	在列表末尾一次性追加另一个序列中的多个值（用新列表扩展原来的列表）
list.pop(index)	移除列表中的某个元素（无参数，则默认移除最后一个元素），并且返回该元素的值
list.remove(obj)	移除列表中某个值的第一个匹配项
list.sort([func])	对原列表进行排序
list.reverse()	反转列表中元素顺序
list.index(obj)	从列表中找出某个值第一个匹配项的索引位置
list.count(obj)	统计某个元素在列表中出现的次数

Avoiding Errors When Working with Lists

Copying a List

Right	Error
>>> my_foods = ['pizza', 'carrot cake']	>>> my_foods = ['pizza', 'carrot cake']
>>> friend_foods = my_foods[:]	>>> friend_foods = my_foods # This doesn't work
>>> print("My favorite foods are: ", my_foods) My favorite foods are: ['pizza', 'carrot cake']	>>> print("My favorite foods are: ", my_foods) My favorite foods are: ['pizza', 'carrot cake']
>>> print("My friend's favorite are:", friend_foods) My friend's favorite are: ['pizza', 'carrot cake']	>>> print("My friend's favorite are:", friend_foods) My friend's favorite are: ['pizza', 'carrot cake']
>>> my_foods.append('cannoli') >>> friend_foods.append('ice cream')	>>> my_foods.append('cannoli') >>> friend_foods.append('ice cream')
>>> print("My favorite foods are: ", my_foods) My favorite foods are: ['pizza', 'carrot cake', 'cannoli']	>>> print("My favorite foods are: ", my_foods) My favorite foods are: ['pizza', 'carrot cake', 'cannoli', 'ice cream']
>>> print("My friend's favorite are:", friend_foods) My friend's favorite are: ['pizza', 'carrot cake', 'ice cream']	>>> print("My friend's favorite are:", friend_foods) My friend's favorite are: ['pizza', 'carrot cake', 'cannoli', 'ice cream']

填空题 2分

在Python中，[填空1]和[填空2]的数据元素是无序的，不能通过位置进行索引。

单选题 1分

下列关于列表推导式的语法结构，哪一个是正确的?

- A [out_exp_res for out_exp in input_list if condition]
- B {out_exp_res in input_list for out_exp if condition}
- C (out_exp_res for out_exp in input_list if condition)
- D out_exp_res for out_exp in input_list if condition

单选题 1分

What is the output of `list(range(2, 6))`?

- A [2, 3, 4, 5]
- B [2, 3, 4, 5, 6]
- C [1, 2, 3, 4, 5]
- D [2, 4, 6]

单选题 1分

如何访问列表中的单个元素?

- A 使用元素的值
- B 使用元素的索引
- C 使用元素的类型
- D 使用元素的长度

填空题 1分

在列表 list1 = ['flower', 'food', 1997, 15.8]中，list1[-1]返回的值是[填空1]。

单选题 1分

下列哪个方法可以将元素添加到列表的末尾?

- A insert()
- B append()
- C remove()
- D pop()

填空题 1分

使用[填空1]方法可以删除列表的末尾元素，并返回该元素。

多选题 1分

关于remove()方法，以下哪些选项是正确的?

- A. remove()方法可以删除列表中所有指定值的元素
- B. remove()方法只删除第一个指定的值
- C. 如果要删除的值可能在列表中出现多次，需要使用循环来删除所有这样的值
- D. remove()方法不能删除列表中的元素

多选题 1分

以下关于range函数生成列表的描述哪些是正确的?

- A list(range(0, 30, 5))生成[0, 5, 10, 15, 20, 25]
- B list(range(0, 10, 3))生成[0, 3, 6, 9]
- C list(range(0, -10, -1))生成[0, -1, -2, -3, -4, -5, -6, -7, -8, -9]
- D list(range(1, 0))生成[1]

Tuples

- Lists work well for storing collections of items that can change throughout the life of a program.
- However, sometimes you'll want to create a list of items that cannot change.
- *Tuples* allow you to do just that.
- **Python refers to values that cannot change as *immutable*(不可变的), and an immutable list is called a *tuple*.**

A tuple looks just like a list except you use parentheses(( )) instead of square brackets([ ]).

Tuples

1. Defining a Tuple

```
>>> person = ('JLY', 'Female', 160, 50)
>>> poker = ('A', '2', '3', '4', '5', '6', '7', '8', '9', '10', 'J', 'Q', 'K')
>>> Options = "a", "b", "c", "d"
>>> Options
('a', 'b', 'c', 'd')
```

```
>>> tup1 = (50,)
>>> tup1
(50,)
>>> tup2 = (50)
>>> tup2
50
>>>
```

- To create an empty tuple, just write empty parentheses.

```
>>> tup_empty = ()
```

- When the tuple contains only one element, U need to add a "," after the element.

```
>>> tup_1 = (50,)
```

补充内容

- Tuple：**不可变序列**，一旦创建不能修改元素；可以做字典 key。

```python
t = (10,20,"abc")
# t[0] = 99 报错，不能修改
```

- **Java**：没有原生 tuple，只能自己写 Class / Record，或者第三方库。
- **C语言**：没有元组，打包多个不同类型的数据，只能用struct 结构体。
- **注意**：tuple 只是本身不可变，但如果里面存可变对象，对象内容可以变：

```python
t = ([1,2], 3)
t[0].append(99) #合法，列表本身可变，tuple只是不能替换里面这个列表对象
```

Tuples

2. Accessing Elements in a Tuple

- Same syntax as accessing list elements

- Example:

```
>>> person = ('Jerry', 'Male', 180, 40)
>>> print (person[0])        # output the first element of the tuple
Jerry
```

Tuples

2. Accessing Elements in a Tuple

- Slicing a Tuple

- Example:

```
>>> Nums_tup = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
>>> print (Nums_tup[1:5])       # slice operation
(1, 2, 3, 4)
>>> print (Nums_tup[2:]) # slice to the last element
(2, 3, 4, 5, 6, 7, 8, 9)
```

Tuples

3. Tuple Join

- Example:

```
>>> tup1 = (12, 34, 56)
>>> tup2 = (78, 90)
>>> tup3 = tup1 + tup2    # Concatenate tuples to create a new tuple
>>> print (tup3)
(12, 34, 56, 78, 90)
>>> tup4 = tup1 * 2       # output tuple twice
>>> print (tup4)
(12, 34, 56, 12, 34, 56)
```

Tuples

4. Writing over a Tuple

- You **can't modify** a tuple:

```
>>> tup1 = (12, 34, 56)
>>> tup1[0] = 100
TypeError: 'tuple' object does not support item assignment
```

- Although you can't modify a tuple, you can assign a new value to a variable that represents a tuple.

```
>>> tup1 = (78, 90)          # redefine the entire tuple
>>> print (tup1)
(78, 90)
```

Tuples

4. Writing over a Tuple

- Although tuples cannot be modified, there **may be exceptions**:

```
>>> tup = (['hi', 3], 5, 7)          # Create a tuple
>>> tup[0][1] = 1          # Modify the value of the first element in the tuple
>>> print(tup)
(['hi', 1], 5, 7)
>>> tup[0].append(4)  # Modify the value of the first element in the tuple again
>>> print(tup)
(['hi', 1, 4], 5, 7)
```

Tuples

5. Delete Tuple

- Elements in a tuple are not allowed to be deleted

```
>>> person = ('Jerry', 'Male', 180, 40)
>>>  del person[3]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object doesn't support item deletion
```

- But you can use the del statement to delete the entire tuple

```
>>> del person
>>> print(person)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'person' is not defined
```

Working with Tuples

Tuples Operator

Operators	Example	Description
+	>>> (1, 2, 3) + (4, 5, 6) (1, 2, 3, 4, 5, 6)	tuple combination (元组组合)
*	>>> ('Hi!',) * 4 ('Hi!', 'Hi!', 'Hi!', 'Hi!')	tuple repeats (元组重复)
in	>>> 3 in (1, 2, 3) True	tuple member operations (元素是否存在于元组中)
in	>>> for x in (1, 2, 3): ... print(x, end=" ") ... 1 2 3	iterative operation (迭代操作)

Working with Tuples

Built-in functions for tuples

Functions	Example	Description
len(tuple)	>>> len((1, 2, 3, 4, 5)) 5	Return the length (the number of items) of a tuple
max(tuple)	>>> max((1, 2, 3, 4, 5)) 5	Return the largest item in an iterable (tuple)
min(tuple)	>>> min((1, 2, 3, 4, 5)) 1	Return the smallest item in an iterable (tuple)

tuple(seq)	>>> tuple([1, 2, 3, 4, 5])
	(1, 2, 3, 4, 5)
	>>> tuple({1, 2, 3, 4, 5})
	(1, 2, 3, 4, 5)
	>>> tuple({'a':1, 'b':2, 'c':3})
	['a', 'b', 'c']
		Convert other sequence type data to tuple

Working with Tuples

Multiple Assignments

- You can use tuples to assign values to multiple variables at once, Example:

```
>>> (x, y, z) = (1, 2, 3)
>>> print(x, y, z)
1 2 3

>>> a, b, c = 3, 4, 5
>>> print(a, b, c)
3 4 5

>>> x, y = y, x    # You can also swap the values of two variables like this
>>> print(x, y)
2 1
```

```
x = input('x=')
y = input('y=')
z = input('z=')
if x < y:
    x, y = y, x
if x < z:
    x, z = z, x
if y < z:
    y, z = z, y
print(x, y, z)
```

Convert between strings, lists, and tuples

str(), tuple(), list()

```
>>> nums_L = [1, 3, 5, 7, 8, 13, 20]
>>> nums_T = tuple(nums_L)
>>> nums_T
(1, 3, 5, 7, 8, 13, 20)
>>> nums_S = str(nums_T)
>>> nums_S
'(1, 3, 5, 7, 8, 13, 20)'
```

Dictionaries

- Dictionary is a standard mapping type, currently only one.
- Dictionary maps hashable values to arbitrary objects.

- Syntax structure of python dictionary:

```
{key1:value1, key2:value2}
```

- value can be any type of data, and does not need to be unique
- key must be unique
- lists, dictionaries or other mutable(可变的) types may not be used as keys.

补充内容

3. dict 字典 {key:value}

- **Python**，内置映射，key 必须可哈希（不可变类型：int、str、tuple可作为key；list 不能做 key），Python3.7+ 字典**保留插入顺序。**
- **Java**： HashMap，无序(对应Python3.6-)；LinkedHashMap保留插入顺序。
- **C语言**：完全没有哈希表，全部手写或者第三方库。

➢ **关键区别**

① Python 可以直接写**字面量{k:v}**（字面量：直接把数据写在代码里，不用调用构造器、不用new，直接写值就创建出对象。）；Java 必须 new HashMap ()。

② Python key 可以是字符串、数字、元组；Java 键必须实现 hashCode () 和 equals ()。

③ Python d["key"]取值，不存在抛异常；.get()安全取值。Java get(key)不存在返回 null。

Dictionaries

1. Create a Dictionary

```
>>> stu1 = {'SN' : 20214987 , 'Name' : 'Jerry' , 'Score' :98 }
```

- The same key is not allowed to appear twice. If the same key is assigned twice during creation, the latter value will overwrite the previous value:

```
>>> info_dict = {'Name' : 'xmj', 'Age' : 17, 'Name' : 'Manni'}
>>> print(info_dict)
{'Name': 'Manni', 'Age': 17}
```

Dictionaries

1. Create a Dictionary

- The values in the dictionary can be any Python object such as *strings*, *numbers*, *tuples*, etc.
- The keys in a dictionary must be immutable(不可变的), so numbers, strings, or tuples can be used, but not lists, dictionaries or other mutable types. Example :

```
>>> info_dict = {['Name'] : 'Zara', 'Age' : 7}
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unhashable type: 'list'
```

Dictionaries

2. Accessing Values in a Dictionary

- Give the name of the dictionary and then place the key inside a set of square brackets, as shown here:

```
>>> stu_dict = {'Name': '王海', 'Age': 18, 'Class': '软件一班'}
>>> print (stu_dict['Name'])
王海
>>> print (stu_dict['Age'])
18
```

- If accessing the data with a key that is not in the dictionary, an error message will be output.

```
>>> print(dict[Score])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'Score' is not defined
```

Dictionaries

2. Accessing Values in a Dictionary

- You can use the get() method of dictionary, even if there is no key, it will not report an error, just return *None*

```
>>> stu_dict = {'Name': '王海', 'Age': 18, 'Class': '软件一班'}
>>> v1 = stu_dict.get('Score')
>>> print(v1)
None
```

Dictionaries

3. Adding New Key-Value Pairs

- Dictionaries are dynamic structures, and you can add new key-value pairs to a dictionary at any time.

```
>>> stu_dict = {'Name': '王海', 'Age': 18, 'Class': '软件一班'}
>>> print(stu_dict)
{'Name': '王海', 'Age': 18, 'Class': '软件一班'}

>>> stu_dict['Score'] = 91.5
>>> print(stu_dict)
{'Name': '王海', 'Age': 18, 'Class': '软件一班', 'Score': 91.5}
```

Dictionaries

4. Modifying Values in a Dictionary

- To modify a value in a dictionary, give the name of the dictionary with the key in square brackets and then the new value you want associated with that key.

```
>>> stu_dict = {'Name': '王海', 'Age': 18, 'Class': '软件一班'}
>>> stu_dict['Age'] = 19                  # update existing entry
>>> stu_dict['School'] = "东北大学"       # add new key-value pairs
>>> print (stu_dict)
{'Name': '王海', 'Age': 19, 'Class': '软件一班', 'School': '东北大学'}
```

Dictionaries

5. Removing Key-Value Pairs

- When you no longer need a piece of information that's stored in a dictionary, you can use the del statement to completely remove a key-value pair.

```
>>> stu_dict = {'Name': '王海', 'Age': 18, 'Class': '软件一班'}
>>> del stu_dict['Name']
>>> print (stu_dict)
{'Age': 18, 'Class': '软件一班'}
```

Dictionaries

5. Removing Key-Value Pairs

```
>>> stu_dict = {'Name': '王海', 'Age': 18, 'Class': '软件一班'}
>>> stu_dict.clear()          # Empty all elements of the dictionary
>>> print (stu_dict)
{}
>>> del stu_dict              # Delete the dictionary, the dictionary does not exist
>>> print (stu_dict)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'stu_dict' is not defined
```

Dictionaries

6. Break a larger dictionary into several lines

```
favorite_languages = {
    'jen': 'python',
    'sarah': 'c',
    'edward': 'ruby',
    'phil': 'python',
    }
```

- It's good practice to include a comma after the last key-value pair as well, so you're ready to add a new key-value pair on the next line.

补充：Dict comprehensions

- The usage of *dict comprehension* and list comprehension is similar, except that the square brackets([ ]) are changed to curly brackets({ }).
- Example:

```
>>> mcase = {'a': 10, 'b': 34, 'A': 7, 'Z': 3}
>>> mcase_freq = {
...  k.lower(): mcase.get(k.lower(), 0) + mcase.get(k.upper(), 0)
                                        # 键值对生成元素表达式
...  for k in mcase.keys()              # 迭代mcase中的key-->k，将 k传入上面的键值对生成表达式中
...  if k.lower() in ['a','b']          # 根据条件过滤元素
... }
>>> print mcase_freq
{'a': 17, 'b': 34}
```

- get方法访问一个不存在的键时，没有异常，返回None，也可以指定一个返回的默认值。
- 这里就是返回0

补充：Set comprehensions

- The usage of ***set comprehension*** and list comprehension is similar, except that the square brackets([ ]) are changed to curly brackets({ }).

- Example:

```
>>> squared = {x**2 for x in [1, 1, 2]}
>>> print(squared)
{1, 4}
```

■ 虽然迭代 list[1,1,2] 给 x 有三个值，但是最终生成集合类型数据，是没有重复数据的，所以少了一个 1

Looping Through a Dictionary

- Because a dictionary can contain large amounts of data, Python lets you loop through a dictionary.
- Dictionaries can be used to store information in a variety of ways;
- therefore, several different ways exist to loop through them.
- You can loop
  - through all of a dictionary's key-value pairs;
  - through its keys;
  - through its values.

Looping Through a Dictionary

1. Looping Through All Key-Value Pairs

```
user_0 = {   # a new dictionary designed to store information about a user on a website.
        'username': 'efermi',
        'first': 'enrico',
        'last': 'fermi',
        }
for k, v in user_0.items():
    print(f"\nKey: {k}")
    print(f"Value: {v}")
```

```
>>> user_0 = {          # a new dictionary
...     'username': 'efermi',
...     'first': 'enrico',
...     'last': 'fermi',
...     }
>>> for key, value in user_0.items():
...     print(f"\nKey: {key}")
...     print(f"Value: {value}")
...

Key: username
Value: efermi

Key: first
Value: enrico

Key: last
Value: fermi
>>>
```

Looping Through a Dictionary

items() Method

- The items() method forms a tuple for each key-value pair in the dictionary, and returns these tuples in a list. Example:

```
>>> stu_dict = {'Name': '王海', 'Age': 18, 'Class': '软件一班'}
>>> print(stu_dict.items())
dict_items([('Name', '王海'), ('Age', 18), ('Class', '软件一班')])
```

- So:

```
>>> for key, value in stu_dict.items():
...     print(key, value)
Name 王海
Age 18
Class 软件一班
```

Looping Through a Dictionary

2. Looping Through All the Keys in a Dictionary

```
favorite_languages = {
        'jen': 'python',
        'sarah': 'c',
        'edward': 'ruby',
        'phil': 'python',
        }
for name in favorite_languages.keys():
    print(name.title())

for name in favorite_languages:
    print(name.title())
```

```
>>> favorite_languages = {
...     'jen': 'python',
...     'sarah': 'c',
...     'edward': 'ruby',
...     'phil': 'python',
...     }
>>> for name in favorite_languages.keys():
...     print(name.title())
...
Jen
Sarah
Edward
Phil
>>>
```

```
>>> name='jiang linying'
>>> name.title()
'Jiang Linying'
>>>
```

- 遍历字典时，会默认遍历所有的键(Key)，所以不用keys()方法也可以
- 但是，显式使用keys()方法，可以让代码更容易理解。

Looping Through a Dictionary

keys() Method

- The keys() method forms a list of all the keys in the dictionary, and return the list. Example:

```
>>> stu_dict = {'Name': '王海', 'Age': 18, 'Class': '软件一班'}
>>> print(stu_dict.keys())
dict_keys(['Name', 'Age', 'Class'])
```

- So:

```
>>> for key in stu_dict.keys():
...     print(key)
Name
Age
Class
```

Looping Through a Dictionary

2. Looping Through All the Keys in a Dictionary

```
favorite_languages = {
        'jen': 'python',
        'sarah': 'c',
        'edward': 'ruby',
        'phil': 'python',
        }

if 'Jerry' not in favorite_languages.keys():
    print("Please take our poll!")
```

- 还可以使用keys() 确定某个人是否接受了调查。
- 例如确定jianglinying是否接受了调查，如果没有，输出“请参加投票！”

Looping Through a Dictionary

3. Looping Through All Values in a Dictionary

```
favorite_languages = {
        'jen': 'python',
        'sarah': 'c',
        'edward': 'ruby',
        'phil': 'python',
        }
print("The following languages have been mentioned:")
for language in favorite_languages.values():
    print(language.title())
```

```
>>> for language in favorite_languages.values():
...     print(language.title())
...
Python
C
Ruby
Python
>>>
```

```
>>> for language in set(favorite_languages.values()):
...     print(language.title())
...
Ruby
C
Python
>>>
```

- 由于字典中的value值是可以重复的，所以为了去重，可以使用set()方法
- 将list转换为set，自然就去重了

```
for language in set(favorite_languages.values()):
    print(language.title())
```

Sets

- A *set* object is an **unordered** collection of **distinct** hashable objects.

- Common uses include
  - membership testing（成员关系测试）
  - removing duplicates from a sequence（去重）
  - computing mathematical operations such as intersection(交集), union(并集), difference(差集), and symmetric difference(对称差集).

补充内容

4. set 集合 {1,2,3}

- **Python**，无序、元素唯一，元素必须可哈希。
- **Java** 对应 **HashSet**
- **C语言**：无内置集合，手写哈希去重。

- 注意：Python 坑：空集合不能写{}，{}是空字典；**空集合必须set()。**

Sets

1. Create a Set

- Sets can be created by several means:
  - Use a comma-separated list of elements within braces:

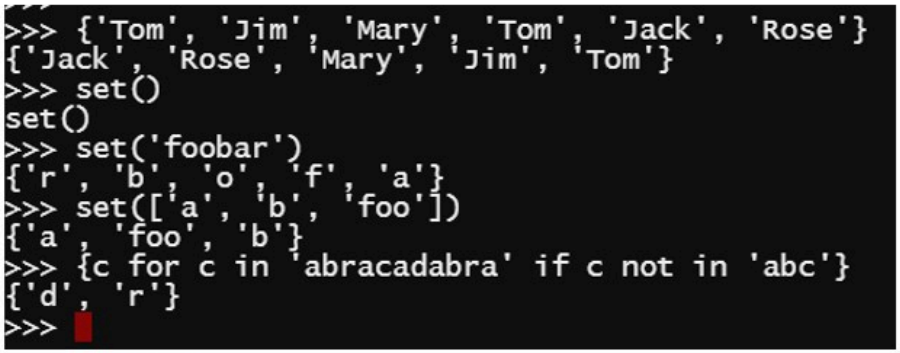
```
>>> {'Tom', 'Jim', 'Mary', 'Tom', 'Jack', 'Rose'}
```

  - Use the type constructor:

```
>>> set()
>>> set('foobar')
>>> set(['a', 'b', 'foo'])
```

  - Use a set comprehension:

```
>>> {c for c in 'abracadabra' if c not in 'abc'}
```



- 创建一个空集合必须用 set() 而不是 { }，因为 { } 是用来创建一个空字典

Working with Sets

Set Operator

- issubset(other)　　#测试set与other的子集、超集关系

Operators	Example	Description
<=	set <= other	Test whether every element in the set is in other.
<	set < other	Test whether the set is a proper subset of other, that is, set <= other and set != other.
>=	set >= other	Test whether every element in other is in the set.
>	set > other	Test whether the set is a proper superset of other, that is, set >= other and set != other.
in	x in s	Test x for membership in s.
not in	x not in s	Test x for non-membership in s.

Working with Sets

Set Operator

- union, intersection, difference, symmetric_difference　　#并、交、差、对称差集

Operators	Example	Description
|	set | other | ...	Return a new set with elements from the set and all others.
&	set & other & ...	Return a new set with elements common to the set and all others.
-	set - other - ...	Return a new set with elements in the set that are not in the others.
^	set ^ other	Return a new set with elements in either the set or other but not both.

Working with Sets

Built-in functions for sets

Functions	Description
len(set)	Return the number of elements in set s.
max(set)	Return the largest item in an iterable (set)
min(set)	Return the smallest item in an iterable (set)
set(seq)	Convert other sequence type data to set

Working with Sets

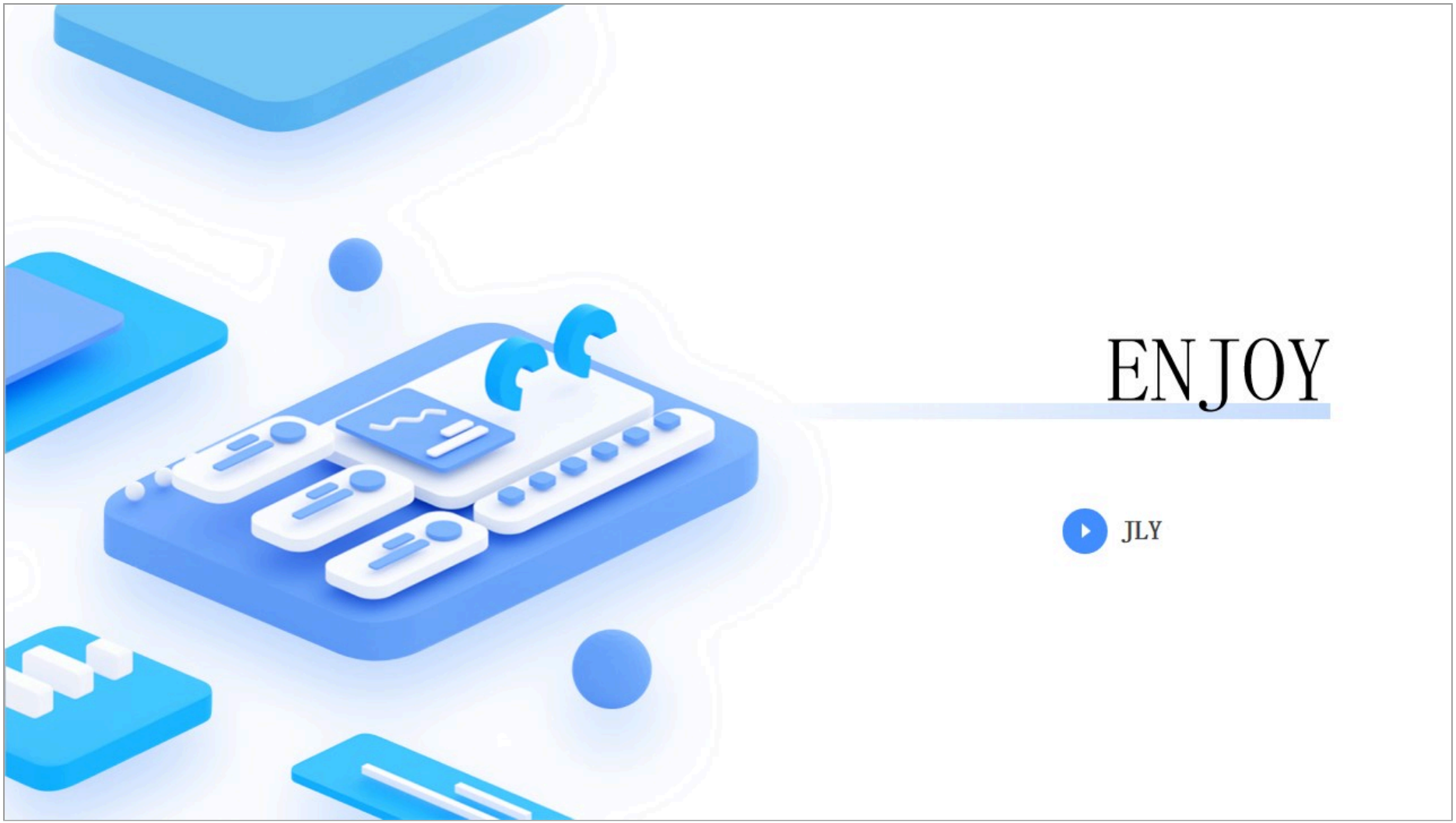
Methods of Sets

Methods	Description
set.add(elem)	Add element elem to the set.
set.remove(elem)	Remove element elem from the set. Raises KeyError if elem is not contained in the set.
set.discard(elem)	Remove element elem from the set if it is present.
set.pop()	Remove and return an arbitrary element from the set. Raises KeyError if the set is empty.
set.clear()	Remove all elements from the set.
set.copy()	Return a shallow copy of the set.
set.isdisjoint(other)	Return True if the set has no elements in common with other. Sets are disjoint if and only if their intersection is the empty set.

多选题 1分

关于元组的定义，以下哪些说法是正确的?

- A 元组是不可变的
- B 元组使用方括号定义
- C 元组可以包含多个元素
- D 定义单个元素的元组时需要在元素后加逗号


ENJOY
JLY